

Monitoreo de temperatura-humedad-ACC

Valentina Padierna Valencia
Leidy Dahiana Espinal Angel
Ana Sofia Zapata Valdes
Julian Jimenez Montalvo
Electrónica Digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	4
7. Comunicación (si aplica)	5
8. Interfaz de usuario (si aplica)	5
9. Procedimiento de prueba	6
10. Manejo de errores y seguridad	6

1. Identificación del proyecto

- **Nombre del proyecto:** Monitoreo de temperatura-humedad-ACC
- **Integrantes del equipo:** Valentina Padierna Valencia, Ana Sofia Zapata Valdes, Julian Jimenez Montalvo y Leidy Dahiana Espinal Angel

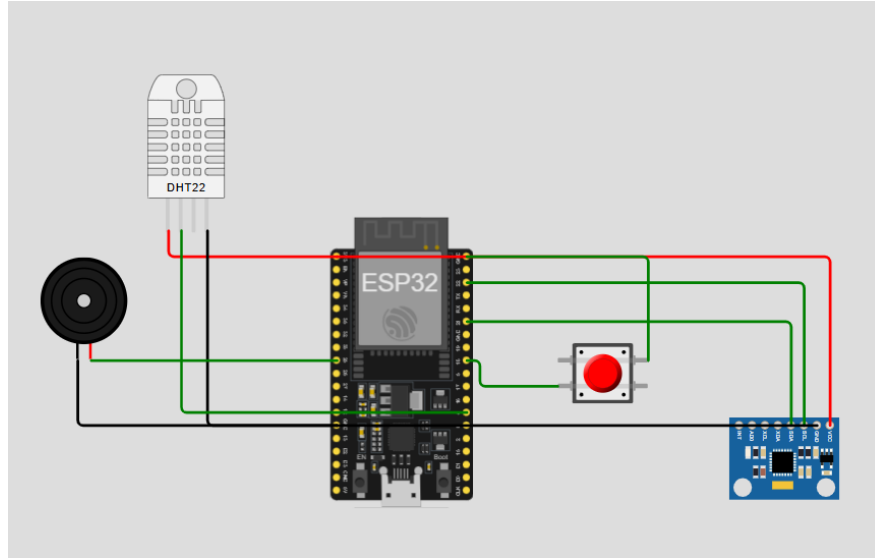
2. Resumen

Este laboratorio consistió en el desarrollo de un sistema de monitoreo biomédico basado en IoT utilizando una ESP32, el sistema integró un sensor DHT11 para medir temperatura y humedad, un sensor MPU6050 para detectar movimiento y un buzzer como sistema de alarma sonora además, se implementó conectividad WiFi para enviar alertas mediante Telegram y visualizar los datos en tiempo real a través de una página web alojada en la ESP32, el programa fue diseñado para identificar condiciones que estuvieran fuera de los rangos ya previamente establecidos en temperatura, humedad y movimiento además de un botón de pánico para emergencias, también se incorporó la manera de realizar consultas mediante telegram conocer el estado actual de los sensores.

3. Descripción del hardware

- **ESP32:** Microcontrolador encargado de ejecutar el minijuego y controlar periféricos del sistema, se emplea debido a su capacidad de procesamiento, disponibilidad de pines GPIO; además, permite utilizar la pantalla OLED, pulsadores y buzzer de manera simultanea y ejecutar el código desarrollado en MicroPython.
- **Sensor DHT11 (GPIO 4):** Sensor digital utilizado para medir temperatura y humedad relativa del ambiente, el módulo proporciona datos digitales al ESP32, permitiendo supervisar variables ambientales y generar alertas cuando los valores superan los umbrales establecidos.
- **Sensor MPU6050 (I2C – GPIO 21 y GPIO 22):** Dispositivo empleado para la detección del movimiento, se comunica mediante protocolo I2C utilizando las líneas SDA y SCL del ESP32 permitiendo identificar estados de reposo, movimiento y movimiento brusco mediante el análisis de aceleración en los ejes x,y,z.
- **Buzzer (GPIO 25):** Elemento utilizado de generar retroalimentación sonora del sistema de alarmas mediante señales PWM, se uso para reproducir sonidos indicando el tipo de alarma detectada por el sistema como temperatura o humedad fuera de rango y si se detecto algun movimiento además del boton de emergencia.
- **Botón de pánico (GPIO 18):** Configurado como entrada digital con resistencia pull-up interna, permite activar manualmente una alerta de emergencia, enviando notificaciones mediante Telegram y activando el buzzer del sistema.

Diagrama de conexión:



4. Descripción del software

El software fue desarrollado en Micropython para la ESP32, con el objetivo de controlar el sistema de monitoreo de temperatura, humedad y movimiento. El programa se va encargar de leer los datos del sensor DTH11, que mide la temperatura y la humedad del ambiente, y tambien del sensor MPU6050, el cual permite detectar si el dispositivo esta en reposo, en movimiento o si se presenta un movimiento brusco. Además, el Código permite conectar la ESP32 a una red Wifi para poder mostrar la información en una página web local. En esta página se pueden observar los valores de temperatura, humedad, estado de movimiento, estado de alarma y los umbrales configurados. También se implementó un Bot de Telegram, el cual enviar alertas cuando alguna variable se sale de los rangos permitidos o cuando se detecta algun tipo de movimiento. El sistema también cuenta con un buzzer, el cual funciona mediante señales PWM y genera diferentes sonidos dependiendo del tipo de alerta que este activa. De esta manera, el proyecto tiene alertas visuales por medio de la pagina web, alertas sonoras por el buzzer y alertas remotas mediante Telegram.

5. Estructura del código

El Código se encuentra organizado en un archivo principal, donde estan todas las funciones necesarias para el funcionamiento del sistema. Primero se importan las librerías necesarias para trabajar con la ESP32, como Pin, PWM e I2C, también se usan librerias para la conexion WiFi, el servidor web, Telegram, operaciones matemáticas y lectura del sensor DTH11. De igual forma se importa la librería del sensor MPU6050, que permite obtener los datos del acelerometro. Luego se configuran los datos principales del sistema, como el nombre de la red WiFi, el token del bot de Telegram, el chat ID y los limites permitidos para temperatura, humedad y movimiento. Despues se inicializan los perifericos, como el sensor DHT11, el

MPU6050 mediante comunicacion I2C, el buzzer por PWM y el boton de panico como entrada digital. Tambien se definen variables para guardar los valores medidos, como temperatura, humedad, estado de movimiento y estado de alarma. Ademas, se utilizan variables booleanas para controlar si una alerta ya fue enviada, esto sirve para que el sistema no mande el mismo mensaje muchas veces seguidas. El programa incluye funciones para conectar el WiFi, enviar mensajes por Telegram, revisar comandos del bot, leer sensores, verificar alertas, crear la pagina web y ejecutar el servidor. Finalmente, todo el sistema funciona dentro de un ciclo principal continuo, donde la ESP32 actualiza constantemente los datos y revisa si existe alguna alarma.

6. Explicación de funciones principales

- **connect_wifi()**: Función encargada de establecer la conexión de la ESP32 con la red WIFI configurada, permitiendo habilitar la comunicación con Telegram y el servidor web.
- **send_telegram()**: Función utilizada para enviar mensajes automáticos de alerta mediante el bot de Telegram cuando alguna variable supera los límites establecidos o se detecta movimiento.
- **send_custom_telegram()**: Función encargada de responder mensajes personalizados enviados por el usuario desde Telegram, mostrando información relacionada con el estado actual del sistema.
- **check_telegram_commands()**: Función utilizada para revisar continuamente los comandos enviados desde Telegram y responder consultas relacionadas con temperatura, humedad, movimiento y umbrales configurados.
- **read_dht()**: Función encargada de realizar la lectura del sensor DHT11 para obtener los valores de temperatura y humedad del ambiente.
- **read_mpu()**: Función utilizada para leer los valores de aceleración del sensor MPU6050 y determinar si el sistema se encuentra en reposo, con movimiento normal o movimiento brusco.
- **check_alerts()**: Función principal del sistema de alarmas encargada de verificar si las variables medidas se encuentran fuera de los límites establecidos, activar el buzzer y enviar las notificaciones correspondientes.
- **webpage()**: Función encargada de generar la página web en formato HTML mostrando en tiempo real las variables medidas y el estado general del sistema.
- **start_server()**: Función encargada de iniciar el servidor web y ejecutar continuamente el monitoreo del sistema, lectura de sensores, revisión de alertas y actualización de la interfaz web.
- **check_panic_button()**: Función utilizada para verificar la activación del botón de pánico y generar una alerta inmediata mediante buzzer y Telegram en caso de emergencia.

7. Comunicación (si aplica)

- Comunicación WIFI

La ESP-32 se conecta automáticamente a una red inalámbrica previamente configurada al iniciar el sistema, donde se establece el nombre de la red y su respectiva contraseña. Esta conexión se realiza gracias a las librerías importadas desde MicroPython. La conexión WIFI permite habilitar las funciones necesarias para el monitoreo y visualización de la información. Además, al establecerse la conexión, la ESP-32 obtiene una dirección IP utilizada posteriormente para el acceso al servidor web.

- Comunicación mediante servidor web

El sistema incorpora un servidor web implementado directamente en la ESP-32. En este servidor se pueden visualizar en tiempo real las variables adquiridas por los sensores, como temperatura, humedad, movimiento y estado de las alarmas. El acceso al servidor web se realiza desde cualquier navegador ingresando la dirección HTTP generada a partir de la dirección IP asignada a la ESP-32 y el puerto correspondiente. Adicionalmente, la página realiza actualizaciones periódicas de los datos, permitiendo la visualización continua de las variables monitoreadas.

- Comunicación mediante Telegram

Se implementó un bot de Telegram para permitir el monitoreo remoto del sistema. El bot permite recibir y responder mensajes enviados por el usuario, posibilitando la consulta de información relacionada con las variables medidas y sus respectivos umbrales desde un dispositivo móvil. Además, cuando alguna variable supera los límites establecidos, la ESP-32 envía automáticamente una notificación de alerta mediante Telegram.

8. Interfaz de usuario (si aplica)

- Interfaz web

Permite visualizar en tiempo real el estado general del sistema desde cualquier navegador conectado a la misma red WIFI. En la página se muestra información correspondiente a las variables medidas por los sensores, como temperatura actual, humedad, estado del sensor de movimiento y estado de las alarmas. Además, cuando se detecta una condición fuera de los límites establecidos, la interfaz muestra mensajes de alerta para advertir al usuario sobre el evento detectado.

- Interfaz de Telegram

Permite la interacción remota con el sistema utilizando el bot de Telegram implementado. El usuario puede solicitar información específica del sistema desde su dispositivo móvil sin necesidad de acceder directamente al servidor web. Adicionalmente, el sistema proporciona retroalimentación automática mediante notificaciones enviadas por el bot cuando alguna variable supera los umbrales establecidos.

9. Procedimiento de prueba

Se realizaron diferentes pruebas con los sensores y módulos conectados al ESP32 con el fin de verificar el correcto funcionamiento del sistema en distintos escenarios. Inicialmente, se comprobó la conexión a la red WIFI y el acceso adecuado a la página web desde un navegador. Posteriormente, se verificó el funcionamiento del sensor DHT11, confirmando la correcta medición de temperatura y humedad.

Adicionalmente, se realizaron pruebas con el sensor MPU6050 mediante movimientos lentos y bruscos para validar la detección de movimiento implementada en el sistema. También se comprobó el funcionamiento del buzzer y el envío de alertas al bot de Telegram cuando las variables superaban los límites establecidos o cuando se activaba el botón de pánico.

Finalmente, se verificó el funcionamiento de los comandos de Telegram y la actualización en tiempo real de las variables mostradas en la interfaz web.

10. Manejo de errores y seguridad

El sistema incluye mecanismos básicos de manejo de errores y seguridad para evitar fallos durante su funcionamiento. Se utilizaron estructuras try-except para controlar posibles errores en la lectura de sensores, la conexión WIFI y la comunicación con Telegram, evitando que el programa se detenga ante fallos temporales.

Adicionalmente, se implementaron límites de temperatura y humedad para detectar condiciones anormales y activar alertas automáticamente. El sistema también incorpora un botón de pánico que permite generar una alarma inmediata en situaciones de emergencia.

Por otra parte, se utilizaron variables de control para evitar el envío repetitivo de alertas, mejorando la estabilidad y el comportamiento general del sistema.